# Standalone 2.5D Swin Video Forensic Visualization Dashboard

## Core Architecture: Frozen Swin-Base (1024-D) + Frequency Branch (768-D) + Temporal Transformer

This notebook contains exclusively the clean class definitions, on-the-fly raw video file streaming loader engines, model state loading hooks, and the 2.5D explainable alert dashboard.

In [1]:
```
# CELL 1: RUNTIME FORENSIC WORKSPACE SETUP
import os
import cv2
import torch
import torch.nn as nn
import torch.fft
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
from torchvision import transforms
from transformers import SwinForImageClassification

sns.set_theme(style="dark")
plt.rcParams['font.family'] = 'sans-serif'

FUSED_DIM = 1792  # 1024 (Swin-Base output pooler) + 768 (Frequency vector channel
EMBED_DIM = 768
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"[STATUS] Forensic visual workspace bound to target hardware device: {DEVICE
```

```
[STATUS] Forensic visual workspace bound to target hardware device: cuda
```

In [2]:
```
class OmniVideoDataset(torch.utils.data.Dataset):
    def __init__(self, video_paths, labels, num_frames=16, transform=None):
        self.video_paths = video_paths
        self.labels = labels
        self.num_frames = num_frames
        self.transform = transform

    def __len__(self):
        return len(self.video_paths)

    def __getitem__(self, idx):
        video_path = self.video_paths[idx]
        label = self.labels[idx]

        cap = cv2.VideoCapture(video_path)
        total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        fps = cap.get(cv2.CAP_PROP_FPS)
```

```
        if total_frames <= 0 or not cap.isOpened() or fps <= 0:
            cap.release()
            return torch.zeros(self.num_frames, 3, 224, 224), label

        # Dynamically sample a coherent 16-frame block across the entire true clip
        sampled_indices = np.linspace(0, total_frames - 1, self.num_frames, dtype=i
        frames_tensors = []
        last_valid_frame = None
        current_frame_idx = 0

        for target_idx in sampled_indices:
            grabbed = True
            while current_frame_idx < target_idx:
                grabbed = cap.grab()
                if not grabbed:
                    break
                current_frame_idx += 1

            if not grabbed:
                fallback_tensor = frames_tensors[-1] if len(frames_tensors) > 0 els
                frames_tensors.append(fallback_tensor)
                continue

            success, frame_bgr = cap.retrieve()
            current_frame_idx += 1

            if success and frame_bgr is not None and frame_bgr.size > 0:
                frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
                img = Image.fromarray(frame_rgb)
                if self.transform:
                    img = self.transform(img)
                frames_tensors.append(img)
            else:
                frames_tensors.append(frames_tensors[-1] if len(frames_tensors) > 0

        cap.release()
        while len(frames_tensors) < self.num_frames:
            frames_tensors.append(frames_tensors[-1])
        return torch.stack(frames_tensors[:self.num_frames]), label
```

In [3]:

```
# CELL 3: PURE SWIN FORENSIC COMPONENT HEADS (MATCHED EXCLUSIVELY TO YOUR BLUEPRINT
class FrequencyBranch(nn.Module):
    def __init__(self, in_channels=3, embed_dim=EMBED_DIM):
        super().__init__()
        self.freq_extractor = nn.Sequential(
            nn.Conv2d(in_channels, 64, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(inplace=True),
            nn.Conv2d(128, embed_dim, kernel_size=3, stride=2, padding=1),
            nn.BatchNorm2d(embed_dim),
            nn.ReLU(inplace=True),
            nn.AdaptiveAvgPool2d((1, 1))
        )
```

```
    def forward(self, x):
        fft_complex = torch.fft.fft2(x, norm="ortho")
        fft_shifted = torch.fft.fftshift(fft_complex, dim=(-2, -1))
        log_magnitude = torch.log(torch.abs(fft_shifted) + 1e-8)
        return torch.flatten(self.freq_extractor(log_magnitude), 1)

class TemporalTransformer(nn.Module):
    def __init__(self, spatial_embed_dim=FUSED_DIM, num_frames=16, num_heads=8, num
        super().__init__()
        self.temporal_cls_token = nn.Parameter(torch.zeros(1, 1, spatial_embed_dim)
        self.positional_encoding = nn.Parameter(torch.randn(1, num_frames + 1, spat

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=spatial_embed_dim,
            nhead=num_heads,
            dim_feedforward=2048,  # Strictly locked to your saved state configurat
            dropout=0.1,
            activation='gelu',
            batch_first=True
        )
        self.temporal_encoder = nn.TransformerEncoder(encoder_layer, num_layers=num

    def forward(self, frame_embeddings):
        B, N, D = frame_embeddings.shape
        cls_tokens = self.temporal_cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, frame_embeddings), dim=1) + self.positional_enco
        return self.temporal_encoder(x)

class OmniDeepfakeDetector(nn.Module):
    def __init__(self, swin_backbone, freq_branch, video_head):
        super().__init__()
        self.swin = swin_backbone
        self.freq_branch = freq_branch
        self.video_head = video_head
        self.feature_norm = nn.LayerNorm(FUSED_DIM)

        # Your Exact Fusion Classifier Head Sequence Layout Parameters
        self.classifier = nn.Sequential(
            nn.Linear(FUSED_DIM, 512),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(512, 2)
        )
    def forward(self, x):
        B, N, C, H, W = x.shape
        images = x.view(B * N, C, H, W)

        swin_output = self.swin(pixel_values=images)
        spatial_features = swin_output.pooler_output  # Native Swin-Base 1024-D str
        freq_features = self.freq_branch(images)      # Native 768-D Spectral matri

        concatenated = torch.cat((spatial_features, freq_features), dim=1)
        features = self.feature_norm(concatenated)
        frame_embeddings = features.view(B, N, -1)
```

```
        transformer_out = self.video_head(frame_embeddings)
        return self.classifier(transformer_out[:, 0])  # Route across [CLS] output
```

In [5]:
```
# CELL 4: INTERCEPT HOOK & STATE RESTORATION
class AttentionHook:
    def __init__(self):
        self.attention_weights = None

    def hook_fn(self, module, input, output):
        # If output is a tuple and the second element exists, grab it
        if isinstance(output, tuple) and output[1] is not None:
            self.attention_weights = output[1].detach().cpu()
        else:
            # FORCE EVALUATION PASS: If need_weights was False, calculate them manu
            with torch.no_grad():
                # Extract original Query, Key, Value vectors passed into the module
                query = input[0]
                key = input[1] if len(input) > 1 else query
                value = input[2] if len(input) > 2 else query

                # Re-invoke self_attn forcing need_weights=True
                _, weights = module(query, key, value, need_weights=True)
                if weights is not None:
                    self.attention_weights = weights.detach().cpu()

SWIN_CHECKPOINT = "microsoft/swin-base-patch4-window7-224"
VIDEO_CHECKPOINT_PATH = "checkpoints/swin_temporal_epoch_15.pth"

print("[SYSTEM] Fetching pretrained parameters via Hugging Face Hub tracking lines.
base_swin = SwinForImageClassification.from_pretrained(SWIN_CHECKPOINT).to(DEVICE)
video_head = TemporalTransformer().to(DEVICE)
freq_branch = FrequencyBranch(embed_dim = 768).to(DEVICE)
freq_branch.load_state_dict(torch.load("checkpoints/freq_branch_standalone.pth", ma

omni_model = OmniDeepfakeDetector(base_swin.swin, freq_branch, video_head).to(DEVIC

if os.path.exists(VIDEO_CHECKPOINT_PATH):
    print(f"[LOAD SUCCESS] Restoring active experimental checkpoints weights: {VIDE
    omni_model.video_head.load_state_dict(torch.load(VIDEO_CHECKPOINT_PATH, map_loc
else:
    print("[ALERT INFO] Target local pth weight tracking vector is invisible. Runni

omni_model.eval()
print("[READY] Model loaded and attention patch applied.")
```

```
[SYSTEM] Fetching pretrained parameters via Hugging Face Hub tracking lines...
Loading weights:   0%|          | 0/449 [00:00<?, ?it/s]
[LOAD SUCCESS] Restoring active experimental checkpoints weights: checkpoints/swin_t
emporal_epoch_15.pth
[READY] Model loaded and attention patch applied.
```

In [14]:
```
def generate_dynamic_video_forensics(video_path, ground_truth_label, model, save_pa
    """
    Natively decodes video tracks frame-by-frame, runs sliding window evaluation pa
    and extracts a clean spatial rollout from Swin without cyclic shift distortion.
    """
```

```
    cap = cv2.VideoCapture(video_path)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    fps = cap.get(cv2.CAP_PROP_FPS)

    if total_frames <= 0 or not cap.isOpened() or fps <= 0:
        raise ValueError(f"Target video structure could not be parsed safely at: {v

    duration = total_frames / fps
    print(f"[METADATA] Frames: {total_frames} | Frame Rate: {fps:.2f} FPS | Duratio

    val_transform = transforms.Compose([
        transforms.Resize((224, 224)),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])

    all_processed_frames = []
    all_original_rgb = []
    last_valid_rgb = None

    # 1. READ ALL FRAME PLUGS WITH HARD BOUNDARY GUARDS
    for frame_idx in range(total_frames):
        grabbed = cap.grab()
        if not grabbed:
            break

        success, frame_bgr = cap.retrieve()
        if success and frame_bgr is not None and frame_bgr.size > 0:
            frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
            last_valid_rgb = cv2.resize(frame_rgb, (224, 224))
            all_original_rgb.append(last_valid_rgb) # Stored securely as uint8
            all_processed_frames.append(val_transform(Image.fromarray(last_valid_rg
        else:
            if last_valid_rgb is not None:
                all_original_rgb.append(last_valid_rgb)
                all_processed_frames.append(val_transform(Image.fromarray(last_vali

    cap.release()
    actual_read_count = len(all_processed_frames)

    if actual_read_count < 16:
        raise ValueError("Video sequence is too short to fulfill base 16 temporal c

    # 2. SLIDING WINDOW EVALUATION LOOP
    window_size = 16
    stride = 8

    window_predictions = []

    for start_idx in range(0, actual_read_count - window_size + 1, stride):
        end_idx = start_idx + window_size
        window_slice = all_processed_frames[start_idx:end_idx]

        video_tensor = torch.stack(window_slice).unsqueeze(0).to(DEVICE)

        with torch.no_grad():
```

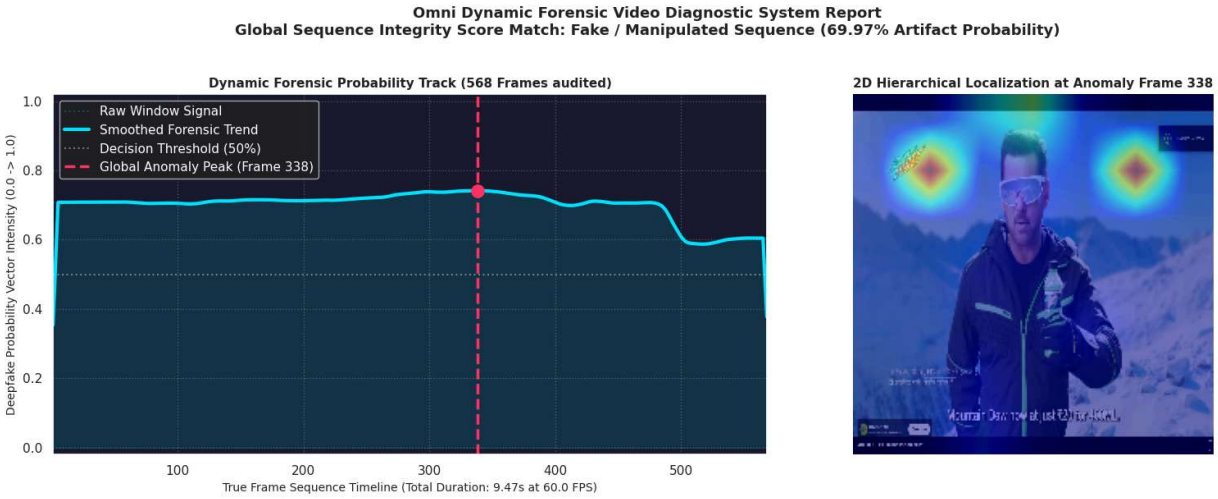
```
            logits = model(video_tensor)
            probs = torch.softmax(logits, dim=1).cpu().numpy()[0]
            window_predictions.append(probs[0])

    overall_fake_severity = np.mean(window_predictions) * 100

    # 3. EXTRACT 2D CROSS-WINDOW SPATIAL ROLLOUT FROM ACTIVE SWIN MEMORY MAP
    # We map the specific frame that triggered the highest smoothed prediction scor
    time_axis = np.arange(1, actual_read_count + 1)
    raw_frame_probabilities = np.interp(
        time_axis,
        np.linspace(1, actual_read_count, len(window_predictions)),
        window_predictions
    )

    kernel_size = 8
    smoothed_probabilities = np.convolve(raw_frame_probabilities, np.ones(kernel_si
    guilty_frame_idx = np.argmax(smoothed_probabilities)

    # Pass ONLY the single guilty frame through Swin to prevent batch-stack collaps
    target_frame_tensor = all_processed_frames[guilty_frame_idx].unsqueeze(0).to(DE

    with torch.no_grad():
        swin_outputs = model.swin(pixel_values=target_frame_tensor, output_attentio

    # CRITICAL FIX: Extract from attentions[-2] (Standard Window) instead of attent
    last_stage_attn = swin_outputs.attentions[-2][0].cpu().numpy()
    mean_spatial_attn = np.mean(last_stage_attn, axis=0)

    # Add residual connection to prevent extreme attention focal collapse
    mean_spatial_attn = mean_spatial_attn + np.eye(mean_spatial_attn.shape[0])
    mean_spatial_attn = mean_spatial_attn / mean_spatial_attn.sum(axis=1, keepdims=

    # Sum over the rows to find the most heavily attended patches
    spatial_rollout = mean_spatial_attn.sum(axis=0)

    grid_size = int(np.sqrt(spatial_rollout.shape[0]))
    raw_spatial_heatmap = spatial_rollout.reshape(grid_size, grid_size)
    spatial_heatmap_resized = cv2.resize(raw_spatial_heatmap, (224, 224))

    if spatial_heatmap_resized.max() != spatial_heatmap_resized.min():
        spatial_heatmap_resized = (spatial_heatmap_resized - spatial_heatmap_resize
                                   (spatial_heatmap_resized.max() - spatial_heatmap_
    else:
        spatial_heatmap_resized = np.zeros_like(spatial_heatmap_resized)

    # 4. DATA PRESENTATION DASHBOARD PANEL AESTHETICS
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6), gridspec_kw={'width_ratio

    # Subplot A: Continuous Anomaly Track Timeline
    ax1.plot(time_axis, raw_frame_probabilities, color="#00E5FF", alpha=0.25, lines
    ax1.plot(time_axis, smoothed_probabilities, color="#00E5FF", linewidth=3, label
    ax1.fill_between(time_axis, smoothed_probabilities, color="#00E5FF", alpha=0.12
    ax1.axhline(y=0.5, color="#FFFFFF", linestyle=":", alpha=0.4, label="Decision T

    ax1.axvline(x=guilty_frame_idx + 1, color="#FF3366", linestyle="--", linewidth=
```

```
    ax1.scatter([guilty_frame_idx + 1], [smoothed_probabilities[guilty_frame_idx]],

    ax1.set_title(f"Dynamic Forensic Probability Track ({actual_read_count} Frames
    ax1.set_xlabel(f"True Frame Sequence Timeline (Total Duration: {duration:.2f}s
    ax1.set_ylabel("Deepfake Probability Vector Intensity (0.0 -> 1.0)", fontsize=1
    ax1.set_xlim(1, actual_read_count)
    ax1.set_ylim(-0.02, 1.02)
    ax1.grid(True, linestyle=":", alpha=0.25, color="#FFFFFF")
    ax1.legend(loc="upper left", facecolor="#222222", labelcolor="white")
    ax1.patch.set_facecolor('#1A1A2E')

    # CRITICAL FIX: Subplot B Heatmap Visualization (Strict uint8 blending)
    clean_display_img_uint8 = all_original_rgb[guilty_frame_idx] # This is pure uin
    heatmap_uint8 = np.uint8(255 * spatial_heatmap_resized)

    color_mapped_heat = cv2.applyColorMap(heatmap_uint8, cv2.COLORMAP_JET)
    color_mapped_heat = cv2.cvtColor(color_mapped_heat, cv2.COLOR_BGR2RGB) # Conver

    # Blend using exact matching depths
    fused_forensic_overlay = cv2.addWeighted(clean_display_img_uint8, 0.55, color_m

    ax2.imshow(fused_forensic_overlay)
    ax2.axis('off')
    ax2.set_title(f"2D Hierarchical Localization at Anomaly Frame {guilty_frame_idx

    verdict = "Fake / Manipulated Sequence" if overall_fake_severity > 50 else "Rea
    fig.suptitle(f"Omni Dynamic Forensic Video Diagnostic System Report\nGlobal Seq
                 fontsize=13, fontweight='bold', y=1.02)

    plt.tight_layout()
    plt.savefig(save_path, bbox_inches='tight', dpi=300)
    plt.show()
    print(f"[COMPLETE] Dynamic visual tracking exported cleanly to path: {save_path
```

In [16]:

```
# CELL 6: COMPILATION TEST INVOCATION INTERFACE CONTEXT
TARGET_VIDEO_PATH = "Desktop 2026.03.21 - 19.39.31.01.mp4"  # Swap with your actual

if os.path.exists(TARGET_VIDEO_PATH):
    generate_dynamic_video_forensics(
        video_path=TARGET_VIDEO_PATH,
        ground_truth_label=0,
        model=omni_model,
        save_path="omni_swin_forensic_report.png"
    )
else:
    print(f"[STATUS LOG] Model layers verified. Replace default token placeholder w
```

```
[METADATA] Frames: 568 | Frame Rate: 60.00 FPS | Duration: 9.47s
```




```
[COMPLETE] Dynamic visual tracking exported cleanly to path: omni_swin_forensic_repo
rt.png
```